第 4 章練習解答　執行緒

> 題號與本章練習題 PDF 一一對應。答案保留成立條件、關鍵推導與必要的反例。

OS04-E01　共享範圍

1. 在抽象 load-add-store 模型中，counter++ 包含讀取、加一、寫回三個步驟；C/C++ 非原子且未同步的衝突存取構成 undefined behavior，不能把此 trace 當成語言所有合法結果。兩個 threads 可能都讀到舊值 0，再分別寫回 1，形成 lost update。
2. thread-local 讓每個 thread 有獨立 counter，消除共享寫入競爭，但其他 threads 無法看到同一累計值，需要額外合併。
3. mutex 讓該 read-modify-write 區間互斥，最終值應為 2。它仍需保證所有存取都遵守同一鎖，否則鎖外存取仍可能 race。

檢查。 競態的判定看同一記憶體位置是否存在未同步的交錯存取。

OS04-E02　並行並不保證加速

1. 加速上限為 1 / ((1-P)+P/N) = 1 / (0.2+0.1) = 1/0.3，約 3.33 倍。
2. 同步、通訊、負載不均、快取一致性、啟動成本與不可平行 I/O 都會增加有效序列比例或額外工作。
3. 上限為 1 / (0.05+0.95/8) = 1/(0.05+0.11875) = 1/0.16875，約 5.93 倍。

檢查。 模型是上界，任何未計成本只能讓實測更差。

OS04-E03　取消與清理

1. Asynchronous cancellation可在半更新共享結構、持鎖或尚未註冊清理時生效，留下不變量破壞或無人釋放資源。
2. Deferred只在明定cancellation points處理，仍要讓那些points位於可清理/rollback的狀態。cond wait取消前會重取得其mutex，再執行cleanup；請求成功不等於完成。
3. 記錄normal/error/cancel的物件所有權，建立handlers前可禁取消；清理B和M每條路徑各一次，避免double free/unlock。成功join後再回收共用參數，有限timeout不證target退出。

檢查。 取消是一個併發狀態轉移，必須有 exit/cleanup 契約。

第 4 章練習解答　執行緒 （續）

OS04-E04　M:N與blocking compensation

1. K1 blocked後9carriers可跑，其餘99個user工作可在它們上輪替。K1…K10全blocked後0可跑，其餘90尚無carrier去發blocking syscalls，停在runtime ready queue。
2. 原上限來自固定10carriers且無補償/async。若補充K11…K20而上限20，可在原10blocked外繼續更多calls；async模型可讓100tasks等待completion而carriers根本不blocked。user-level等待與kernel blocked不同。
3. M:1僅1carrier真blocking時其他user threads不能由kernel另排；1:1每thread有kernel entity，一個blocked不阻止其他ready者，多核平行仍受core/ready數限制。

檢查。 模型前提、數值、同步邊界與生命週期同時成立。

OS04-E05　Create、join與TLS

1. Main先初始化input，create成功後不讀worker output；Work活到成功join後，再讀42。Sleep10ms不是完成同步，worker可能還未排到；create成功也不保證worker尚未開始。
2. pthread_join僅用joinable target，處理return error number；Win wait需成功signaled而非timeout/WAIT_FAILED，thread handle仍CloseHandle但close不殺thread；Java start才開新thread，直接run普通呼叫，join InterruptedException不是完成證明。
3. 皆不安全，前者物件已失效，後者worker仍可能用它；同步不能復活失效物件。Detached需另一完成/所有權協議。
4. 每thread counter各0→1，join後可由main合併2；TLS跨函式且隨worker長存，pool重用不會每job自動清空。job狀態需明確重設，不能把thread identity當job identity。

檢查。 模型前提、數值、同步邊界與生命週期同時成立。

第 4 章練習解答　執行緒 （續）

OS04-E06　Amdahl、DAG與false sharing

1. T8=20+80/8=30ms；speedup100/30=10/3；efficiency(10/3)/8=5/12；P無限時speedup≤100/20=5。額外總overhead10ms時T8=40、speedup2.5。
2. 若80operations各1cycle、另20各10，T1=280；只改善前者8倍後210，speedup4/3。不能把數量比例偷換時間比例。
3. leaf1…8，pair3/7/11/15，quartet10/26，root36。8leaf+7merge work15，最長路徑4；TP≥max(15/P,4)，speedup≤15/4。平均parallelism不是超過3.75processors一律無收益的斷點。
4. 不同int且無其他衝突可沒有語言data race，卻同line可false sharing。分隔對齊line可降低此干擾，未實測不得填改善倍數；若本來是同counter則仍需同步，padding不會修data race。

檢查。 模型前提、數值、同步邊界與生命週期同時成立。

OS04-E07　Pool與serial queue的等待環

1. 兩個workers被A/B佔著等待，children ready卻無空worker執行；資源等待形成starvation deadlock，與task資料依賴是否為DAG不同。
2. 用已明定支援help-join的runtime，使等待者執行ready dependencies；或不在pool worker內blocking等同pool child，改continuation/外部協調；調大pool只可解此例，不能保證任意巢狀等待。
3. 後者要等前者離Q，前者又等後者，self-wait。Serial queue只序列該queue工作，不自動保護不同queues的共享物件。
4. private partials3/7/11/15，merge10/26→36；plain shared sum+=無同步不等於reduction。浮點重排還可能改最後bits，必須指定誤差/operation契約。

檢查。 模型前提、數值、同步邊界與生命週期同時成立。

第 4 章練習解答　執行緒 （續）

OS04-E08　Cancel、fork與signal的不同身分

1. 否，只排請求；target可disabled或尚未到deferred cancellation point。Main成功join才能依相應契約知道target完成，不能先free其參數。
2. wait原子釋M等待；取消被處理前重新取得M；LIFO先free B一次，再unlock M一次。正常路徑也配對pop/清理；註冊期間可禁取消，抽象不變量破壞時不得任意到cancellation point。
3. 不會。可能把破壞的不變量暴露給下一holder。應在穩定邊界才允取消或設計真正rollback；async任意點取消尤其危險。
4. child只保留caller T1，記憶體中T2的lock可仍顯示持有者卻無T2可解；exec前須遵守async-signal-safe限制與整體atfork設計。fork child繼承parent的signal dispositions與呼叫者T1的signal mask，但child pending signals起始為空。Signal disposition通常process共享，mask perthread；process-directed signal不是每thread各執行一次handler，同步fault則指故障thread。

檢查。 模型前提、數值、同步邊界與生命週期同時成立。

OS04-E09　Windows/Linux固定案例的物件計數

1. CreateProcessW已含initial thread，再CreateThread一次合計2thread，各有ETHREAD/KTHREAD/TEB與獨立stacks/registers；共同1process/address space。若再建兩次會3threads，不能漏初始者。
2. 普通fork有2task、2mm、2page tables，各fd table可複製entries仍引用原open descriptions。指定sharing clone有2task與各自kernel stack，但1mm/PT、1files、1sighand；CLONE_THREAD使共同TGID，不合併task/TID。
3. 不一定，這些共享contexts卻沒有CLONE_THREAD的TGID語意。v3.2 THREAD要求SIGHAND，SIGHAND要求VM；pthread實作還有TLS/termination等其他機制，不能只憑四flags完整等同。
4. 同backing可供兩task存取，但相衝寫入/publication仍需同步與生命期；共享address space不提供所有操作atomicity。

檢查。 模型前提、數值、同步邊界與生命週期同時成立。

第 4 章練習解答　執行緒 （續）

原創驗算程式附錄: join_result.cpp

```
// POSIX Pthreads example: Work remains alive until a successful join.
// Build: g++ -std=c++17 -Wall -Wextra -Werror -pthread join_result.cpp -o join_result
#include <pthread.h>
#include <cstdio>

struct Work { int input; int output; };

void* worker(void* argument) {
    Work* work = static_cast<Work*>(argument);
    work->output = 3 * work->input;
    return nullptr;
}

int main() {
    Work work{14, 0};
    pthread_t thread;
    int error = pthread_create(&thread, nullptr, worker, &work);
    if (error != 0) {
        std::fprintf(stderr, "pthread_create error=%d\n", error);
        return 1;
    }
    error = pthread_join(thread, nullptr);
    if (error != 0) {
        std::fprintf(stderr, "pthread_join error=%d\n", error);
        return 1;
    }
    // No read of output occurs before the worker has completed.
    std::printf("%d\n", work.output);
    return work.output == 42 ? 0 : 2;
}
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

第 4 章練習解答　執行緒 （續）

原創驗算程式附錄: thread_models.py

```
"""Exact arithmetic and sequential teaching models; not hardware benchmarks."""
from fractions import Fraction


def amdahl(baseline,serial_fraction,processors,overhead=0):
    if baseline<=0 or not 0<=serial_fraction<=1 or processors<1 or overhead<0:
        raise ValueError("invalid time model")
    time=baseline*(serial_fraction+(1-serial_fraction)/processors)+overhead
    speedup=baseline/time
    return time,speedup,speedup/processors


def reduce_tree(values):
    """Power-of-two leaves, one unit per leaf and binary merge."""
    if not values or len(values)&(len(values)-1):
        raise ValueError("nonempty power-of-two input required")
    level=list(values);levels=[level];work=len(level);span=1
    while len(level)>1:
        level=[level[i]+level[i+1] for i in range(0,len(level),2)]
        levels.append(level);work+=len(level);span+=1
    return {"levels":levels,"result":level[0],"work":work,"span":span}


def carriers(initial,blocked,replacement=False,maximum=None):
    maximum=initial if maximum is None else maximum
    if initial<1 or maximum<initial or blocked<0:
        raise ValueError("invalid carrier configuration")
    configured=min(initial+blocked,maximum) if replacement else initial
    if blocked>configured:raise ValueError("more blocked carriers than exist")
    return {"configured":configured,"blocked":blocked,"runnable":configured-blocked}


if __name__=="__main__":
    for overhead in (0,10):
        print("Amdahl:",amdahl(100,Fraction(1,5),8,overhead))
    print("Reduction:",reduce_tree(list(range(1,9))))
    print("Fixed:",carriers(10,10))
    print("Compensated:",carriers(10,10,True,20))
```

參考資料與署名

Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 9th Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。
https://github.com/xinhi6716/computer-science-self-study